

The Dual-Write Problem

 systemdesignschool.io/fundamentals/cache-consistency



The Consistency Problem

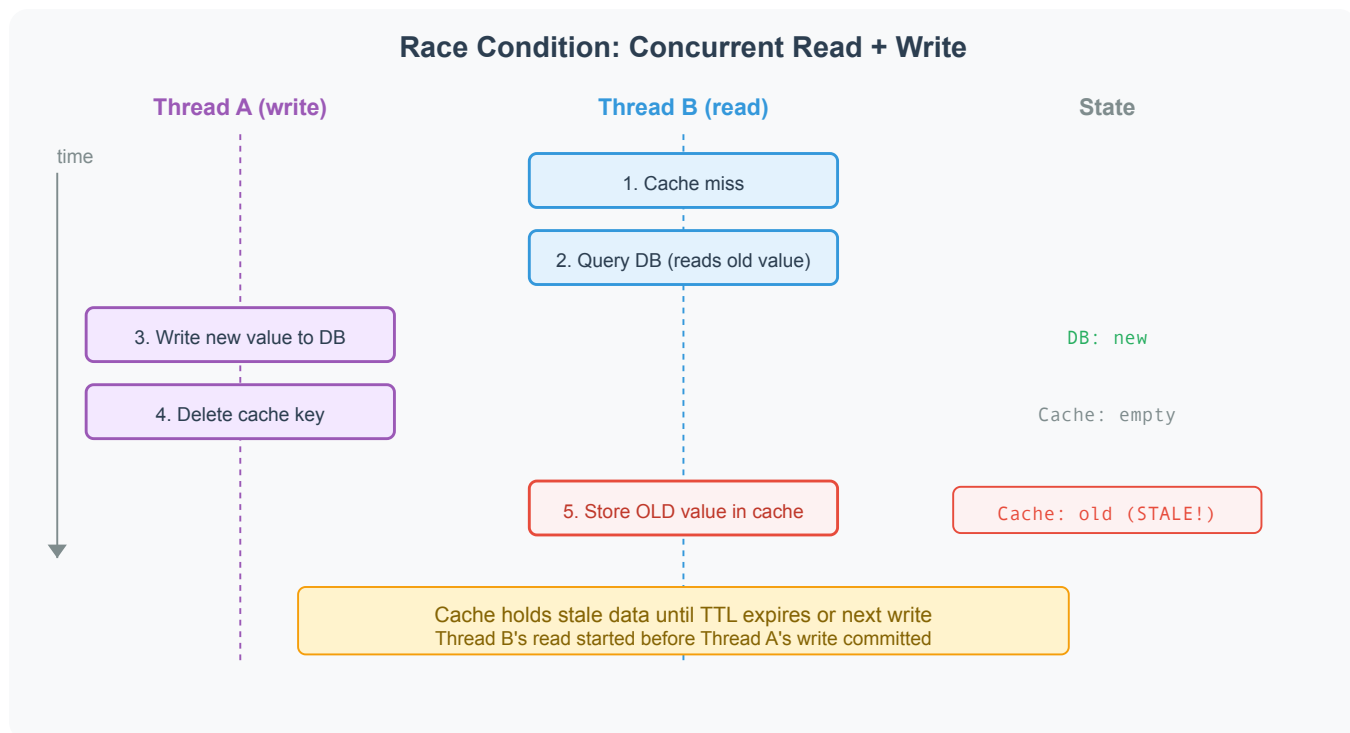
Write-around deletes the cache key after writing to the database. Write-through updates both synchronously. Both seem correct. But when multiple requests hit the system concurrently, subtle race conditions leave stale data in the cache—sometimes indefinitely.

Any system with a cache and a database maintains two copies of the same data. A write must update both. The question is: in what order, and what happens if something fails between the two operations?

Consider an application using write-around. The write flow is: (1) update the database, (2) delete the cache key. Two things can go wrong.

Crash between steps. The application updates the database but crashes before deleting the cache key. The cache now holds the old value. Every subsequent read returns stale data until the entry expires via TTL or another write invalidates it.

Concurrent read during write. This is more subtle and harder to prevent.



The sequence: Thread A updates the database (user email changes to `new@example.com`). Thread B reads the cache, finds a miss, and queries the database—but its query started before Thread A's write committed. Thread B gets the old value (`old@example.com`). Thread A deletes the cache key. Thread B writes the old value into the cache. The cache now contains stale data that persists until the next invalidation.

This race exists regardless of which write pattern you use. It happens whenever a read-miss and a write overlap in time.

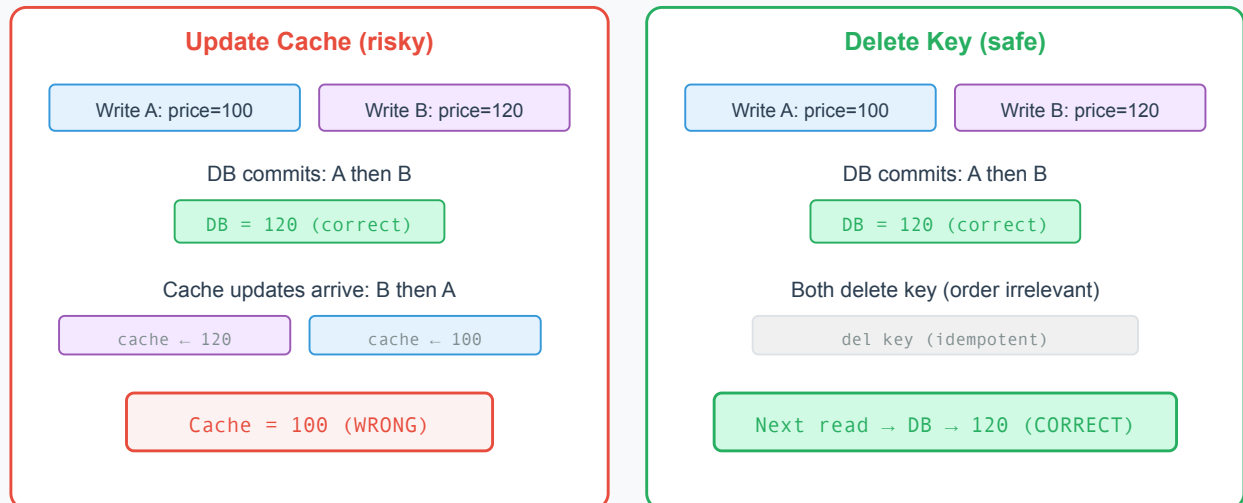
Delete vs Update: Why Invalidation Wins

When a write occurs, the application has two choices for the cache:

1. **Update the cache** with the new value (write-through approach)
2. **Delete the cache key** and let the next read repopulate (write-around approach)

Updating sounds better—the cache is always fresh. But updating introduces a new race condition.

Delete vs Update: Out-of-Order Writes



Deletion is idempotent and order-independent. Updates require ordering guarantees.



Suppose two writes arrive in quick succession: Write A sets `price=100`, Write B sets `price=120`. Both update the database (B wins because it commits last). Both update the cache. But if Write A's cache update arrives after Write B's, the cache ends up with `price=100` while the database has `price=120`. The cache is wrong. A TTL will eventually expire the stale entry, but without one the error persists until the next write.

Deletion avoids this entirely. If both writes delete the key, the cache is simply empty. The next read fetches the correct value (120) from the database. Deletion is **idempotent**—deleting a key that's already gone is a no-op. Updating is order-dependent—the last update wins, and "last" in a distributed system is hard to guarantee.

This is why many systems prefer delete-on-write (write-around) over update-on-write for simplicity.

Mitigation Strategies

No pattern eliminates the consistency gap completely in a distributed system. But several strategies reduce the window and impact.

Short TTLs. Every cache entry expires after a bounded time (seconds to minutes). Even if a stale write sneaks in, it self-corrects when the TTL expires. The tradeoff: shorter TTLs mean more cache misses and higher database load.

Write-through for critical data. For data where staleness is unacceptable (account balances, inventory counts), use write-through. The synchronous double-write reduces the inconsistency window significantly, though partial failures can still cause divergence. The cost is write latency.

Delayed double-delete. After writing to the database and deleting the cache key, wait a short interval (e.g., 500ms) and delete the key again. This catches the race where a concurrent read repopulates the cache with stale data during the write. The second delete clears the stale entry.

Version stamps. Attach a version number or timestamp to each cache entry. On write, increment the version. On cache population, only store the value if the version is newer than what's already cached. This prevents old values from overwriting new ones. The complexity is maintaining consistent version ordering across services.

The Consistency Spectrum

Different data types tolerate different staleness windows.

Data Type	Acceptable Staleness	Pattern
Account balance	None	Write-through or skip cache
Product price	Seconds	Write-around + short TTL
User profile	Minutes	Write-around + moderate TTL
Recommendation scores	Hours	Write-behind + long TTL
Static content	Days	CDN with explicit purge

The right answer is rarely "make everything consistent." Strong consistency costs write latency and system complexity. Choose the staleness tolerance for each data type and apply the simplest pattern that meets it.

This leads to a hands-on lab implementing a write-behind worker. You'll see write coalescing in action—and what happens when the worker crashes before flushing.